12.6 CRUD Operations with MongoDB

Once the model is set up, you can perform CRUD operations in a manner familiar to any Laravel developer using Eloquent. To create a new product, you can use the `create()` method. Reading all records from the collection can be achieved with `Product::all()`. Updating a document involves retrieving it with `find()` and then calling `save()` after modifying its fields. Deleting can be done using the `destroy()` method. Here is an example of each:

```
// Create
Product::create(['name' => 'SSD 1TB', 'price' => 149, 'category' => 'storage']);

// Read
$products = Product::all();

// Update
$product = Product::find($id);
$product->price = 129;
$product->save();

// Delete
Product::destroy($id);
```

This seamless integration makes working with MongoDB intuitive for developers already comfortable with Laravel's Eloquent ORM.

12.7 Using MongoDB Aggregation and Queries

MongoDB offers powerful querying capabilities, including support for advanced conditions and aggregation pipelines. With Laravel's MongoDB package, many of these features are accessible through familiar query builder syntax. You can use operators such as `$gte`, `$lte`, `$in`, and `$or` to perform advanced searches. For example:

```
// Price greater than 100
Product::where('price', '>', 100)->get();

// Category in a list
Product::whereIn('category', ['storage', 'memory'])->get();
```

These queries allow you to filter and retrieve data efficiently without needing to write raw MongoDB queries.

12.8 Real-World Use Case: Product Inventory